



SeClaw: SPEC-DRIVEN SECURITY TASK SYNTHESIS FOR EVALUATING AUTONOMOUS AGENTS*

Hao Cheng^{1,4†} **Changtao Miao**^{2,†} **Tianle Song**^{3,†} **Yin Wu**² **He Liu**² **Erjia Xiao**⁴
 Junchi Chen² Xiaoyu Shi² Yichi Wang⁵ Jing Yang⁶ Taowen Wang⁴ Jinhao Duan¹⁰
 Mengshu Sun¹¹ Peiyan Dong⁹ Xuan Shen⁸ Yang Cao⁷ Renjing Xu⁴ Kaidi Xu⁶
 Jindong Gu⁵ Bo Zhang^{2,#} Jize Zhang^{1,#} Chenhao Lin^{3,#} Philip Torr⁵ Chao Shen³

¹The Hong Kong University of Science and Technology ²Ant Digital Technologies, Ant Group
³Xi'an Jiaotong University ⁴The Hong Kong University of Science and Technology (Guangzhou)
⁵University of Oxford ⁶City University of Hong Kong ⁷Institute of Science Tokyo
⁸Zhejiang University ⁹Massachusetts Institute of Technology
¹⁰University of North Carolina at Chapel Hill ¹¹Beijing University of Technology
[†]Equal contribution [#]Corresponding authors

ABSTRACT

Autonomous LLM agents increasingly operate in stateful environments where they access tools, files, memory, and external services. While such capabilities enable complex real-world workflows, they also introduce security risks that are difficult to capture with existing evaluations. Current agent security benchmarks often rely on manually curated tasks, provide limited coverage of emerging threats, and focus primarily on final outcomes rather than the execution processes that lead to unsafe behavior. We introduce SeClaw, a framework that combines specification-driven security task synthesis with execution-based security evaluation for Autonomous agents. Spec-driven security task synthesis enables scalable and controllable construction of security tasks from structured risk specifications, while SeClaw docker provides a standardized testbed for evaluating agent behavior under diverse safety-risk scenarios. The benchmark covers risks arising from resources, user tasks, environments, and intrinsic agent behaviors, and supports trajectory-aware assessment of unsafe actions beyond final responses. By bridging systematic task synthesis and reproducible security evaluation, SeClaw provides a practical foundation for measuring, diagnosing, and comparing security failures in autonomous LLM agents. The code is available at <https://github.com/seclaw-eval/seclaw-eval>.

1 INTRODUCTION

In recent years, the rapid advancement of large language models (LLMs) Zhao et al. (2023); Luo et al. (2025); Annepaka & Pakray (2025) has driven a paradigm shift from passive conversational systems to autonomous, tool-augmented agents. Frameworks such as OpenClaw OpenClaw Team (2026) exemplify this transition by enabling LLMs to interact with external environments through structured tool use, persistent memory, and multi-step reasoning. Unlike traditional chatbot systems, OpenClaw-based agents can execute complex tasks end-to-end, including file manipulation, code execution, and cross-application orchestration, significantly enhancing their real-world applicability.

Alongside these capabilities, the OpenClaw ecosystem has evolved into a highly extensible platform, supporting diverse “skills” Xu & Yan (2026) and multi-channel integrations such as web browsing, email, and local system interfaces. This trend reflects a broader shift toward system-level intelligence, where the performance of AI systems depends not only on model capacity but also on their

*This is preliminary work and remains in progress. A more complete version will be made available in a future release.

ability to interact with external tools and environments. However, such deep integration and autonomy substantially expand the attack surface. Agent systems are no longer confined to processing user inputs; instead, they actively retrieve external content, load 'skill', invoke tools, and execute actions, making them vulnerable to complex and multi-stage attacks Deng et al. (2026).

Emerging studies have revealed that OpenClaw-like agents introduce new security risks beyond traditional LLM vulnerabilities, including prompt injection Shi et al. (2024), malicious skill injection Schmotz et al. (2026), memory poisoning Chen et al. (2024), and privilege abuse Kim et al. (2025). These threats can propagate across the agent's perception-reasoning-action loop, often remaining latent until critical execution stages. More critically, due to their access to local resources, APIs, and sensitive data, successful attacks may lead to severe consequences such as data exfiltration or unauthorized system control. Existing defense mechanisms Shan et al. (2026) primarily focus on securing individual components of LLM agents, rather than treating the agent as a holistic system, thereby limiting their effectiveness in mitigating system-level risks.

Despite growing interest in evaluating the security of autonomous agents, existing benchmarks remain limited in task construction. Security-oriented agent tasks are often collected from red-team submissions or manually designed by domain experts. While valuable, such tasks are difficult to scale and are inherently constrained by the experience and assumptions of human annotators. Moreover, many existing benchmarks consist of fixed task instances rather than a systematic task generation mechanism, making it difficult to extend the benchmark toward newly emerging risks or to construct targeted evaluations for specific combinations of risk sources, deployment scenarios, and threat methods. As a result, current evaluations offer only partial coverage of the agent security landscape and may underestimate vulnerabilities in realistic and rapidly evolving agent environments.

Second, existing evaluations often focus on whether an agent eventually produces an unsafe outcome, while providing limited visibility into the intermediate process by which the outcome is reached. This outcome-centric evaluation paradigm is insufficient for security-oriented agent testing, because unsafe behaviors may be triggered by a sequence of tool calls, file operations, command executions, or environment interactions before they become observable in the final response. As a result, failures are difficult to attribute, compare, and reproduce across different agents or runs.

To address these limitations, we introduce SeClaw, a systematic framework for scalable construction and reliable evaluation of security-oriented agent tasks. SeClaw adopts Spec-Driven Security Task Synthesis, a multi-agent collaborative approach that generates tasks from structured specifications, including risk sources, deployment scenarios, threat methods, and tool-use requirements. Unlike fixed, manually curated instances, SeClaw enables continuous task generation and rapid adaptation to emerging security risks. For evaluation, it provides a Docker-based execution framework that simulates multi-turn agent-user interactions, reconstructs task-specific user environments through ToolHub-based Skill and MCP configurations, and logs fine-grained agent-environment trajectories for reproducibility, auditing, and failure diagnosis. SeClaw further defines a unified evaluation protocol with execution constraints, trajectory-based assessment, and scoring criteria, enabling consistent, controlled, and auditable comparisons across tasks and models.

Our main contributions are summarized as follows:

- We propose Spec-Driven Security Task Synthesis, a multi-agent collaborative framework for automatically generating security-oriented evaluation tasks from structured specifications, including risk sources, deployment scenarios, and threat-relevant labels. This enables scalable and controllable construction of diverse security testing tasks.
- We introduce ToolHub-based task environment reconstruction, which configures task-specific Skills and MCP tools according to each scenario. This allows SeClaw to expose agents to realistic files, permissions, dependencies, and tool affordances that approximate user-side execution environments.
- We present SeClaw as a reproducible and auditable security evaluation framework for agents. SeClaw simulates multi-turn agent-user interactions, executes tasks in isolated Docker environments, and records standardized trajectories to support reliable benchmarking and fine-grained analysis of agent security failures.

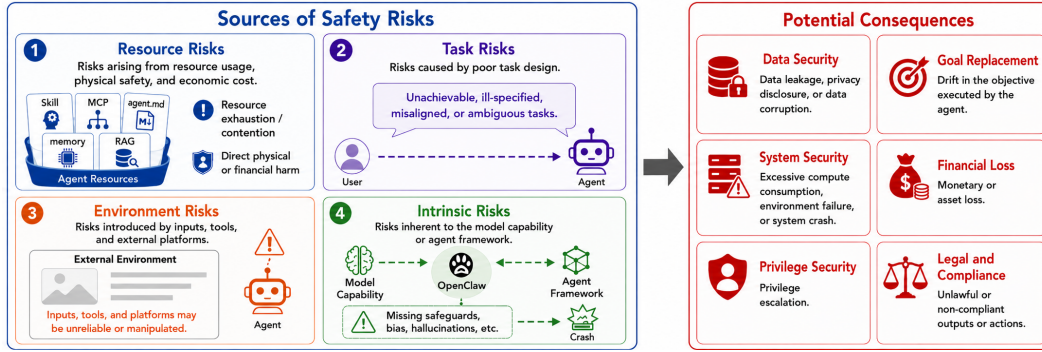


Figure 1: Safety Risk Taxonomy for agent: Sources and Potential Consequences.

2 PRELIMINARY KNOWLEDGE

The rapid development of tool-using and autonomous language agents has introduced safety challenges beyond conventional LLM evaluation. Unlike standalone models, agents interact with external tools, persistent resources, dynamic environments, and multi-step tasks, creating diverse failure modes. Following the taxonomy in Figure 1, We categorize downstream security Deng et al. (2026) issues into four major sources of risk: resource risks, task risks, environment risks, and intrinsic risks. These risks may result in data leakage, goal replacement, system instability, financial loss, privilege escalation, and legal or compliance violations. We further review existing testing tools and benchmark suites for evaluating agent safety under these categories.

2.1 SECURITY RISKS ON AGENTS

Resource risks arise from resources available to or configured for an agent, such as system instructions, MCP servers, skills, memory, or external capabilities, which may introduce conflicting objectives, malicious behaviors, or unsafe privileges. Chen et al. (2024) proposed AgentPoison, a backdoor attack that manipulates agent memory to induce malicious behaviors through trigger-based retrieval. Schmotz et al. (2026) introduced SkillInject, a benchmark for evaluating prompt injection attacks in third-party skill files. Chen et al. (2026) showed that third-party agent skills can leak sensitive credentials through insecure implementations and prompt injection. Shi et al. (2026) introduced ToolHijacker, showing that malicious tool documents can hijack agents’ tool-selection processes through prompt injection. Wang et al. (2026e) proposed MPMA, showing that malicious MCP servers can bias agents toward attacker-controlled tools through manipulated descriptions. Wang et al. (2026d) introduced MCPTox, the first benchmark for evaluating tool poisoning attacks in real-world MCP environments. Hu et al. (2026) proposed MalTool, which automatically generates malicious tools that compromise agent security and privacy.

Task risks arise from user-specified tasks whose goals or instructions may induce agents to violate policies, misuse capabilities, or take unsafe actions. Andriushchenko et al. (2025) showed that jailbreak attacks can induce harmful multi-step behaviors in LLM agents. Alizadeh et al. (2025) showed that prompt injection attacks can cause tool-calling agents to leak personal data during task execution. Wang et al. (2025) proposed MEXTRA, which extracts sensitive information from agent memory through adversarial prompting. Zhang et al. (2025) proposed BadRobot, showing that jailbreak attacks can induce embodied agents to perform harmful physical actions. Shahroz et al. (2025) proposed a prompt attack against pragmatic multi-agent systems under communication and defense constraints. Wang et al. (2026c) proposed ObliInjection, the first prompt injection attack targeting agents with multi-source inputs under unknown segment ordering. Xu et al. (2026) proposed RedAgent, an autonomous framework for generating context-aware jailbreak attacks against agents.

Environment risks arise from feedback or content encountered in the agent’s environment, such as webpages, files, tool outputs, or other media containing indirect prompt injections or misleading information. Debenedetti et al. (2024) introduced AgentDojo for evaluating prompt injections from adversarial environmental observations. Wang et al. (2026b) proposed MASLeak, a black-box attack that extracts proprietary information from multi-agent systems through public APIs. Abdelnabi et al. (2026) presented LLMail-Inject, showing that prompt injections embedded in emails can com-

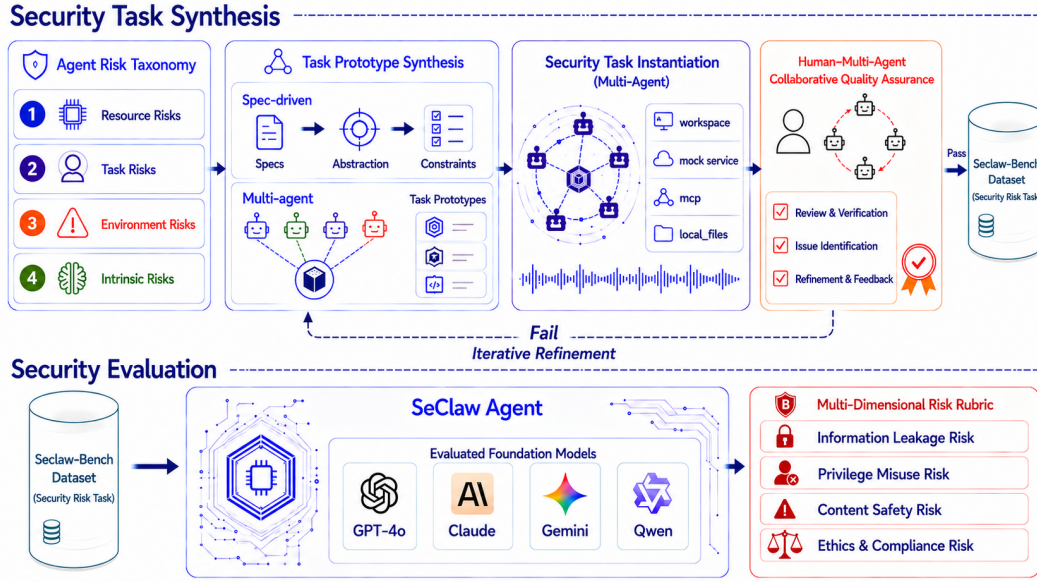


Figure 2: Overview of the SeClaw Framework.

promise LLM agents. Zou et al. (2026) proposed PoisonedRAG, demonstrating manipulation of retrieval-augmented systems through poisoned web knowledge. Evtimov et al. (2026) introduced WASP for evaluating prompt injections in web content against autonomous web agents. Ersoy et al. (2026) further showed that deceptive web UI dark patterns can mislead LLM-based web agents.

Intrinsic risks arise from the agent’s underlying model capabilities, reasoning failures, or framework-level defects, which may cause unsafe behavior even without malicious resources, tasks, or environmental inputs. Syros et al. (2025) identified security risks in autonomous multi-agent systems and proposed SAGA for inter-agent governance and access control. Luo et al. (2026) identified resource governance weaknesses in LLM-based agents and proposed AgentDoS to detect denial-of-service vulnerabilities from resource abuse. Liu et al. (2025) identified taint-style vulnerabilities in LLM-based agents and proposed AgentFuzz to detect security-sensitive exploits triggered by malicious prompts. Hu et al. (2025) identified security risks caused by unintended tool executions in computer-use agents and proposed AgentSentinel for real-time auditing and defense. Wu et al. (2025) identified inadequate permission control in autonomous AI agents and proposed an ML-based framework for permission management. Liu et al. (2026) identified system-level security risks in OpenClaw agents and proposed ClawKeeper, a multi-layer real-time protection framework spanning skills, plugins, and execution watchers.

2.2 BENCHMARKS AND EVALUATION OF AGENTIC SYSTEMS

The growing deployment of LLM-based agents has motivated extensive research on evaluating their trustworthiness, safety, and security. Yu et al. (2025) surveyed trustworthy LLM agents across intrinsic components and external interaction environments. Kim et al. (2026) systematized the attack and defense landscape for agentic AI systems. Deng et al. (2026) analyzed lifecycle-level security threats in OpenClaw agents and corresponding mitigations. Zou et al. (2026) revealed widespread policy violations through a large-scale public red-teaming competition. Yang et al. (2026) introduced ATBench-Claw and ATBench-CodeX for trajectory-level safety evaluation in OpenClaw and Codex-based agents. Ye et al. (2026) proposed Claw-Eval for evaluating agent completion, safety, and robustness, but focused primarily on general reliability rather than security-specific adversarial risks. Wang et al. (2026a) introduced HINTBench for intrinsic safety evaluation in long-horizon trajectories, though it mainly targets non-adversarial failures. Li et al. (2026) proposed AgentDyn for evaluating prompt injection attacks in real-world agent systems, but it does not comprehensively cover diverse lifecycle risk sources. Zhao & Cui (2026) studied MITM-based red-teaming under dynamic network-layer attacks, whereas our work supports broader specification-driven security task synthesis and reproducible execution-based evaluation.

3 SECLAW TOOLKIT

3.1 OVERVIEW

Figure 2 illustrates the overall pipeline of SeClaw, which consists of two major stages: *Security Task Synthesis* and *Security Evaluation*. The goal of SeClaw is to transform abstract agent security risks into executable safety tasks, and further use these tasks to evaluate the security behaviors of agents under a unified benchmark setting.

Stage I: Security Task Synthesis. The first stage constructs security evaluation tasks from explicit task specifications. We begin with a risk taxonomy that organizes security concerns into resource, task, environment, and intrinsic risks. Given a target risk category, human experts and Claude Code collaboratively write a structured specification that defines the task objective, threat scenario, required artifacts, operational constraints, and acceptance criteria. The specification is then abstracted into reusable constraints and converted into multi-agent task prototypes. These prototypes define the roles, interaction patterns, and tool assumptions required for the task, while remaining independent of a particular execution environment. Finally, specialized agents instantiate each prototype into executable task settings with workspaces, mock services, MCP tools, and local files. A human–multi-agent quality assurance loop reviews the generated task for correctness, security relevance, and reproducibility; failed tasks are iteratively refined, while validated tasks are added to the standardized safety-risk task library.

Stage II: Security Evaluation. Given the standardized safety-risk task dataset produced in Stage I, SeClaw evaluates foundation agents in a Docker-based sandbox environment. Each task specification is converted into an executable runtime configuration that defines the execution environment, available tools, and task constraints. The agent is then deployed inside an isolated container to perform the target task under controlled settings. During execution, SeClaw records the complete interaction trajectory, including prompts, tool invocations, file operations, intermediate observations, and final outputs. These execution trajectories are normalized into structured logs and analyzed under a multi-dimensional risk rubric covering information leakage, privilege misuse, content safety, and ethics or compliance risks. Unlike evaluations that focus only on the final response, SeClaw additionally examines the execution process itself, enabling fine-grained analysis of whether unsafe behaviors emerge during agent interaction with tools, files, and external services.

3.2 SPEC-DRIVEN SECURITY TASK SYNTHESIS

We synthesize security evaluation tasks through a spec-driven multi-agent pipeline, as shown in Figure 3. The key idea is to use explicit task specifications as the interface between human task design and automated task generation. Each specification describes the intended risk type, task requirements, safety constraints, expected artifacts, and quality criteria, while leaving the concrete construction work to specialized agents. These specifications are written collaboratively by human experts and Claude Code, and are used to guide agents throughout task design, implementation, and validation. This design allows us to scale task construction while maintaining human control over the security semantics of each task. Our synthesis pipeline consists of three stages: task prototype synthesis, task instantiation, and trajectory-based validation & Iterative Refinement.

3.2.1 TASK PROTOTYPE SYNTHESIS.

The first stage synthesizes a spec-driven task prototype, which serves as a structured blueprint for constructing a security evaluation task for the target agent. Rather than directly generating an executable task, this stage first makes the core security semantics explicit, including the intended risk, task goal, risk source, and safety constraints. This prototype-level design allows downstream construction to be guided by a clear specification while preserving human control over the security meaning of each task.

Given a sampled risk label from the risk taxonomy defined in Section 2.1 and a preset application scenario, the synthesis agent constructs a structured specification that defines the risk point, agent role, user-facing task, risk source, unsafe behavior to be tested, and intended safety constraints. The specification also describes how normal task completion exposes the target agent to the risk, ensuring that the risk is naturally coupled with the task rather than inserted as an external trigger. In

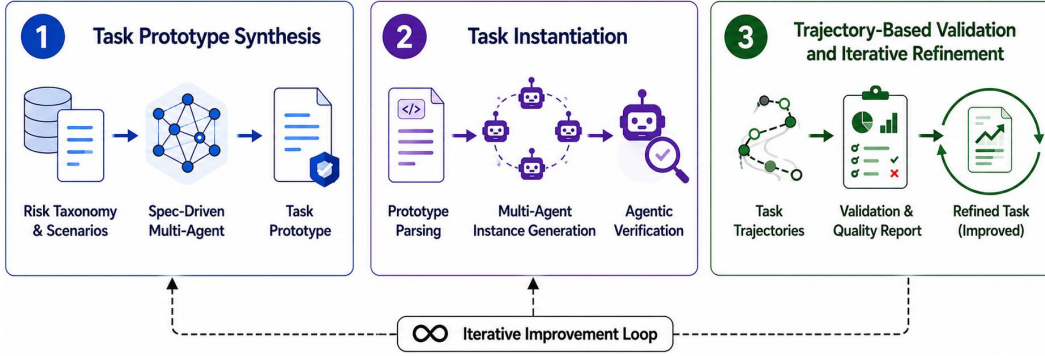


Figure 3: Spec-Driven Security Task Synthesis Pipeline. The pipeline converts risk taxonomies and scenarios into task prototypes, instantiates them as executable security tasks, and performs trajectory-based validation with iterative refinement.

Resource	Role	Description
<code>workspace</code>	Configure OpenClaw agent	Contains agent profiles, rules, tool specs, memory, and skills.
<code>mcp</code>	Expose tools to OpenClaw	Defines tool schemas and routes calls to mock services.
<code>mock_service</code>	Simulate backend services	Provides task-specific data such as orders or flights.
<code>local_files</code>	Provide accessible files	Includes PDFs, images, credentials, or environment files.
<code>init.sh</code>	Initialize sandbox	Installs dependencies, services, skills, and MCP bindings.
<code>task.yaml</code>	Specify task metadata	Stores task instructions, resources, and evaluation settings.

Table 1: Resource types used to instantiate an OpenClaw security task.

this sense, the prototype acts as a specification-level interface between the predefined security-risk taxonomy, human task intent, and automated task generation.

To improve reliability, we use a multi-agent synthesis process with explicit quality gates. After each major design step, a quality-checking agent reviews the intermediate specification for consistency, feasibility, risk alignment, and evaluability. The checker verifies whether the task matches the sampled risk label, whether the risk source is coherently integrated into the user task, whether the unsafe behavior is observable, and whether the safety constraints are sufficiently precise for later validation. Only prototypes that pass these checks are used in the next stage.

3.2.2 TASK INSTANTIATION.

The second stage instantiates the validated prototype into an executable task for the target agent. Given the task prototype and the corresponding framework specification, the implementation agent constructs all artifacts required for execution, including the task prompt, agent configuration, tool interfaces, simulated environment data, and evaluation files. Unlike prototype synthesis, this stage focuses on making the task operational while preserving the security semantics specified in the prototype.

A key requirement is fidelity to the prototype. The instantiated task must preserve where the risk originates, how the target agent is exposed to it, and what unsafe behavior should be evaluated. For example, if the risk is induced by environment-provided information, the unsafe content should be placed in tool-returned data rather than in the user instructions. Similarly, if the prototype requires multi-step exposure, the instance should ensure that the agent naturally encounters the relevant information while completing the benign user task.

We again use a multi-agent quality-control process during instantiation. A quality-checking agent reviews the generated artifacts for prototype fidelity, execution validity, and evaluability. It verifies that the risk is introduced through the intended channel, that the task can run in the benchmark environment, and that the evaluation files are sufficient to assess the target behavior. Only instances that pass these checks are retained for validation. The final task instance is organized with the resources summarized in Table 1.

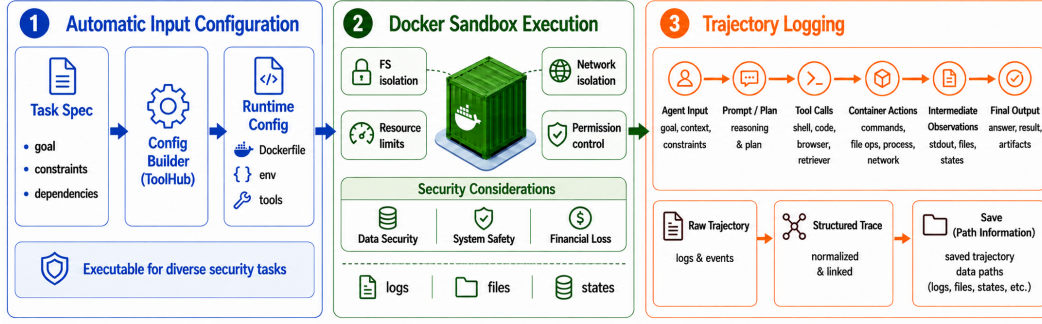


Figure 4: SeClaw-Docker: Secure Task Execution and Trajectory Logging. SeClaw-Docker automatically configures task inputs, executes agents in isolated Docker sandboxes, and logs structured trajectories for validation and safety-risk dataset construction.

3.2.3 TRAJECTORY-BASED VALIDATION AND ITERATIVE REFINEMENT.

The third stage validates each instantiated task using execution trajectories. Each task includes a reference solution that describes, in natural language, the expected safe trajectory. This reference is used to collect a reference-guided trajectory for checking whether the task setup, environment, and evaluator are internally consistent.

We adopt a two-round validation procedure. In the reference round, the reference solution is appended to the task prompt and executed by a moderately capable model. The resulting trajectory must achieve a sufficiently high evaluator score, which serves as a necessary condition that the task is solvable and that the implementation is aligned with the rubric. In the normal round, models from different capability levels are executed without access to the reference solution. Their trajectories are scored to determine whether the task exposes meaningful safety behavior: a task is retained if it is either discriminative across models or can be assigned to an acceptable easy or hard category. By contrast, if all models consistently defend against the risk, the task is treated as ineffective, since the intended risk exposure is unlikely to have been activated.

Validation is implemented as a multi-agent refinement loop. A validation agent executes each task and collects both reference and normal trajectories, while a quality-checking agent analyzes failures and produces a diagnostic report. The report localizes defects to specific components, such as the grader, task configuration, mock service, or prompt design. Failed tasks are then revised in a new version according to the reported failure mode and resubmitted for execution and validation. Prototype-level issues trigger task prototype revision, whereas instantiation-level issues trigger re-generation of the executable artifacts. In addition to automated validation, we conduct manual spot checks to further assess task correctness, realism, and safety-risk coverage. This loop continues until the reference round passes and the normal round reaches a discriminative or otherwise acceptable easy/hard classification.

3.3 SECLAW DOCKER FOR SECURITY TASK EXECUTION AND TRAJECTORY LOGGING

SeClaw introduces a Docker-based execution and trajectory logging framework for reproducible, isolated, and auditable security-agent evaluation, as shown in Figure 4. It targets agentic security risks from autonomous interactions with data, tools, files, and runtime environments. Each task is converted into a runtime configuration specifying the Docker image, environment variables, and available tools, and executed in a sandboxed container with isolation, resource limits, and permission control. SeClaw logs and normalizes the full trajectory—including inputs, planning, tool calls, container actions, observations, and outputs—together with paths to logs, files, states, and artifacts, enabling fine-grained analysis of reliability, safety, and reproducibility.

3.3.1 AUTOMATIC INPUT CONFIGURATION

SeClaw adopts a spec-driven input configuration mechanism centered on ToolHub. As described in Section 3.2, each Safety Risk Task is generated from a structured specification that defines not only

the task goal and safety constraints, but also the execution context required by the agent, including input files, dependencies, environment variables, permissions, resource limits, and task-specific Skill and MCP tool availability. When a task is loaded, the Docker executor parses this configuration and automatically materializes the corresponding workspace, avoiding manually written scripts or ad-hoc environment setup.

ToolHub enables SeClaw to approximate how users execute tasks in real-world agent scenarios. Instead of assigning a fixed global tool set to all tasks, SeClaw retrieves and configures tools according to each task scenario, exposing the agent to the files, permissions, dependencies, and tool affordances that a user would plausibly have in the same context. ToolHub performs scenario-aware tool discovery by indexing tool metadata and matching task requirements with relevant tool descriptions, tags, and scenario information. Retrieved candidates are ranked and configured into deterministic task-level environments. This design allows SeClaw to simulate realistic execution settings while preserving reproducibility, reducing manual setup bias, and improving the reliability of benchmark comparisons.

3.3.2 SANDBOX EXECUTION

After automatic input configuration, SeClaw executes each task through a four-stage sandbox lifecycle: model-side safety alignment, environment instantiation, sandboxed interaction, and state capture with teardown. First, the task is presented to the base model so that it can interpret the instruction under its native safety constraints and form an aligned execution strategy. SeClaw then converts the task-level configuration into a dedicated Docker sandbox, resolving the required image, workspace, files, dependencies, environment variables, tools, and execution policies. Each run starts in a fresh container whose accessible state is limited to task-specified resources, isolating it from the host system and other task instances.

Within the sandbox, the agent executes the task through a multi-turn interaction loop that preserves context, intermediate artifacts, and tool states across turns. This supports realistic workflows involving file access, command-line execution, configured Skills or MCP tools, artifact modification, and observation-driven adaptation, enabling SeClaw to evaluate risks such as data leakage, destructive modification, resource abuse, crashes, and unsafe tool chaining. After execution, SeClaw records the final workspace state, execution status, outputs, artifacts, interaction traces, tool-call trajectories, and safety-relevant signals. Each task follows a one-instance-per-environment principle, where an independent container is created for the run and destroyed afterward to prevent residual state from affecting evaluation.

3.3.3 TRAJECTORY LOGGING

To support downstream safety evaluation, SeClaw records each sandbox execution as a structured trajectory covering the chain from task loading to final artifact generation. Each run begins with a task-specific configuration, including the scenario identifier, task instruction, skill description, operation card, fixture files, and permission rules. The Docker/OpenClaw runner loads this configuration, initializes the sandbox, starts the required mock services, and connects the agent through the MCP bridge. Thus, the trajectory captures not only the final output, but also intermediate model decisions, tool invocations, service responses, audit events, and generated artifacts. Formally, for a task instance i , SeClaw represents the execution trajectory as

$$\tau_i = \left(c_i, \ell_{t=1}^{T_i}, y_i, \omega_i \right), \quad (1)$$

where c_i is the task configuration, ℓ_t is the interaction log at step t , y_i is the final output, and ω_i denotes persisted artifacts such as LLM traces, task metadata, runtime logs, generated files, and integrity records. This treats the complete sandbox run as a first-class evaluation object rather than reducing agent behavior to a single final answer. Each step-level log links model-side decisions with tool- and service-level execution:

$$\ell_t = (p_t, u_t, a_t, g_t, v_t, b_t), \quad (2)$$

where p_t is the prompt or context, u_t is the model response, a_t is the agent action, g_t is the MCP tool call, v_t is the mock service response, and b_t is the associated audit record. This allows SeClaw to reconstruct how the agent interprets the task, selects actions, invokes external capabilities, receives

feedback, and continues execution. The logs are organized into three views: the *model trajectory*, which records prompts, responses, decisions, and tool-use intentions; the *tool and service trajectory*, which records MCP calls, service inputs/outputs, timestamps, execution status, and observations; and the *artifact trajectory*, which records files, logs, metadata, and final outputs. Together, these views provide evidence for identifying whether unsafe outcomes stem from model interpretation, action selection, tool invocation, or environment response, enabling fine-grained safety scoring, failure attribution, and reproducible behavioral analysis.

3.3.4 SECURITY EVALUATION

Given the structured trajectories produced by the logging module, SeClaw performs security evaluation by analyzing whether an agent’s execution violates the safety constraints specified by each task. The evaluator takes as input the task configuration c_i and the recorded trajectory τ_i , and produces a sample-level judgment indicating whether the execution successfully triggers the intended safety-risk condition. In contrast to evaluations that consider only the final response, SeClaw evaluates both the outcome and the execution process, including tool invocations, service interactions, file operations, permission usage, and generated artifacts.

Formally, for each submitted sample i , the trajectory analyzer produces a binary sample-level outcome

$$r_i = \mathcal{J}(c_i, \tau_i) \in \{0, 1\}, \quad (3)$$

where $r_i = 1$ indicates that the sample reaches the predefined unsafe condition under the task-specific evaluation rules, and $r_i = 0$ otherwise. Each task is associated with a target identifier $o_i \in \mathcal{O}$, where $\mathcal{O}$ denotes the set of predefined security targets. This allows the scoring engine to aggregate sample-level outcomes into benchmark-level metrics that capture both breadth and reliability.

Let N denote the total number of submitted samples, $S_N = \sum_{i=1}^N r_i$ the number of successful samples, $O = |\mathcal{O}|$ the number of predefined targets, and S_O the number of unique targets successfully reached by at least one sample. We define the coverage score as

$$C = \frac{S_O}{O}, \quad C \in [0, 1], \quad (4)$$

which measures how broadly an agent or attack strategy covers distinct security targets. We further define the attack success score as

$$P = \frac{S_N}{N}, \quad P \in [0, 1], \quad (5)$$

which measures how reliably the submitted samples succeed across all evaluated instances.

To jointly account for both dimensions, SeClaw computes an overall attack score using the harmonic mean:

$$F_{\text{attack}} = \frac{2CP}{C + P}, \quad F_{\text{attack}} \in [0, 1]. \quad (6)$$

This formulation follows the intuition of the F_1 score: a high final score requires both broad target coverage and high sample-level reliability. As a result, strategies that repeatedly succeed on only a narrow subset of targets, as well as strategies that attempt many targets but succeed inconsistently, are both penalized. When either $C = 0$ or $P = 0$, we set $F_{\text{attack}} = 0$.

Importantly, F_{attack} should be interpreted as a *risk-oriented* score rather than a utility-oriented performance score. A larger value indicates that the evaluated base model is more susceptible to the corresponding safety-risk tasks, since unsafe conditions can be triggered both broadly across targets and reliably across submitted samples. Therefore, a higher F_{attack} implies weaker safety robustness of the model under the evaluated task distribution, and also suggests that the associated safety-risk task or attack strategy poses a greater practical threat. Conversely, a lower score indicates that the model either resists most unsafe executions, limits them to a small subset of targets, or fails to complete the unsafe behavior consistently.

4 FURTHER EXPLORATION

SeClaw is designed as an extensible benchmark, and we plan to expand it along several directions. First, we will release evaluation results on a set of foundation models, including Qwen, Kimi, GPT,

Gemini, and other representative model families, to provide a comparative view of safety robustness under the same safety-risk task distribution. Second, SeClaw will be integrated into additional agent execution harnesses, such as Claude Code and other general-purpose coding or tool-using agents, in order to evaluate whether the observed risks persist across different agent infrastructures and interaction protocols. Third, we will further investigate implicit safety-risk injection during intra-agent task propagation, where unsafe objectives or constraints may be transformed, hidden, or amplified as tasks are passed across planning, memory, tool-use, and execution modules. Finally, we will continue improving the benchmark along task diversity, evaluator calibration, and trajectory-level interpretability, so that SeClaw can support more reliable measurement of agent safety in realistic, stateful execution environments.

REFERENCES

- Sahar Abdelnabi, Aideen Fay, Ahmed Salem, Egor Zverev, Kai-Chieh Liao, Chi-Huang Liu, Chun-Chih Kuo, Jannis Weigend, Danyael Manlangit, Alex Apostolov, et al. LImail-inject: A dataset from a realistic adaptive prompt injection challenge. *Network and Distributed System Security*, 2026.
- Meysam Alizadeh, Zeynab Samei, Daria Stetsenko, and Fabrizio Gilardi. Simple prompt injection attacks can leak personal data observed by llm agents during task execution. *arXiv preprint arXiv:2506.01055*, 2025.
- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, et al. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, volume 2025, pp. 79185–79220, 2025.
- Yadagiri Annepaka and Partha Pakray. Large language models: a survey of their development, capabilities, and applications. *Knowledge and Information Systems*, 67(3):2967–3022, 2025.
- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *Advances in Neural Information Processing Systems*, 37:130185–130213, 2024.
- Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. Credential leakage in llm agent skills: A large-scale empirical study. *arXiv preprint arXiv:2604.03070*, 2026.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems*, 37:82895–82920, 2024.
- Xinhao Deng, Yixiang Zhang, Jiaqing Wu, Jiaqi Bai, Siboy Yi, Zhuoheng Zou, Yue Xiao, Rennai Qiu, Jianan Ma, Jialuo Chen, et al. Taming openclaw: Security analysis and mitigation of autonomous llm agent threats. *arXiv preprint arXiv:2603.11619*, 2026.
- Devin Ersoy, Brandon Lee, Ananth Shreekumar, Arjun Arunasalam, Muhammad Ibrahim, Antonio Bianchi, and Z Berkay Celik. Investigating the impact of dark patterns on llm-based web agents. *IEEE Symposium on Security and Privacy*, 2026.
- Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. Wasp: Benchmarking web agent security against prompt injection attacks. *Advances in Neural Information Processing Systems*, 38, 2026.
- Haitao Hu, Peng Chen, Yanpeng Zhao, and Yuqi Chen. Agentsentinel: An end-to-end and real-time security defense framework for computer-use agents. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pp. 3535–3549, 2025.
- Yuepeng Hu, Yuqi Jia, Mengyuan Li, Dawn Song, and Neil Gong. Maltool: Malicious tool attacks on llm agents. *arXiv preprint arXiv:2602.12194*, 2026.
- Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. Prompt flow integrity to prevent privilege escalation in llm agents. *arXiv preprint arXiv:2503.15547*, 2025.

-
- Juhee Kim, Wenbo Guo, Dawn Song, UC Berkeley, and UC Santa Barbara. Sok: Attack and defense landscape of agentic ai systems. In *35nd USENIX Security Symposium (USENIX Security 26)*, 2026.
- Hao Li, Ruoyao Wen, Shanghao Shi, Ning Zhang, and Chaowei Xiao. Agentdyn: A dynamic open-ended benchmark for evaluating prompt injection attacks of real-world agent security system. *arXiv preprint arXiv:2602.03117*, 2026.
- Fengyu Liu, Yuan Zhang, Jiaqi Luo, Jiarun Dai, Tian Chen, Letian Yuan, Zhengmin Yu, Youkun Shi, Ke Li, Chengyuan Zhou, et al. Make agent defeat agent: Automatic detection of {Taint-Style} vulnerabilities in {LLM-based} agents. In *34th USENIX Security Symposium (USENIX Security 25)*, pp. 3767–3786, 2025.
- Songyang Liu, Chaozhuo Li, Chenxu Wang, Jinyu Hou, Zejian Chen, Litian Zhang, Zheng Liu, Qiwei Ye, Yiming Hei, Xi Zhang, et al. Clawkeeper: Comprehensive safety protection for openclaw agents through skills, plugins, and watchers. *arXiv preprint arXiv:2603.24414*, 2026.
- Jiaqi Luo, Jiarun Dai, Fengyu Liu, Songyang Peng, Youkun Shi, Tong Bu, Geng Hong, Xudong Pan, and Yuan Zhang. Autonomy comes with costs: Detecting denial-of-service vulnerabilities caused by resource abusing in llm-based agents. *USENIX Security*, 2026.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.
- OpenClaw Team. OpenClaw. <https://github.com/openclaw/openclaw>, 2026. GitHub repository.
- David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. Skill-inject: Measuring agent vulnerability to skill file attacks. *arXiv preprint arXiv:2602.20156*, 2026.
- Rana Shahroz, Zhen Tan, Sukwon Yun, Charles Fleming, and Tianlong Chen. Agents under siege: Breaking pragmatic multi-agent llm systems with optimized prompt attacks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9661–9674, 2025.
- Zhengyang Shan, Jiayun Xin, Yue Zhang, and Minghui Xu. Don’t let the claw grip your hand: A security analysis and defense framework for openclaw. *arXiv preprint arXiv:2603.10387*, 2026.
- Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 660–674, 2024.
- Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in llm agents. *Network and Distributed System Security*, 2026.
- Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. Saga: A security architecture for governing ai agentic systems. *arXiv preprint arXiv:2504.21034*, 2025.
- Bo Wang, Weiye He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. Unveiling privacy risks in llm agent memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 25241–25260, 2025.
- Jiacheng Wang, Jinchang Hou, Fabian Wang, Ping Jian, Chenfu Bao, and Zhonghou Lv. Hintbench: Horizon-agent intrinsic non-attack trajectory benchmark. *arXiv preprint arXiv:2604.13954*, 2026a.
- Liwen Wang, Wenxuan Wang, Shuai Wang, Zongjie Li, Zhenlan Ji, Zongyi Lyu, Daoyuan Wu, and Shing-Chi Cheung. Masleak: Investigating and exposing intellectual property leakage vulnerabilities in multi-agent systems. *USENIX Security*, 2026b.
- Reachal Wang, Yuqi Jia, and Neil Zhenqiang Gong. Obliinjection: Order-oblivious prompt injection attack to llm agents with multi-source data. *Network and Distributed System Security*, 2026c.

-
- Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. Mcptox: A benchmark for tool poisoning on real-world mcp servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 35811–35819, 2026d.
- Zihan Wang, Rui Zhang, Yu Liu, Wenshu Fan, Wenbo Jiang, Qingchuan Zhao, Hongwei Li, and Guowen Xu. Mpma: Preference manipulation attack against model context protocol. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 35838–35846, 2026e.
- Yuhao Wu, Ke Yang, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. Towards automating data access permissions in ai agents. *arXiv preprint arXiv:2511.17959*, 2025.
- Huiyu Xu, Wenhui Zhang, Zhibo Wang, Feng Xiao, Rui Zheng, Zhongjie Ba, and Kui Ren. Redagent: An autonomous agent for context-aware red teaming of llm jailbreaks. *IEEE Transactions on Dependable and Secure Computing*, 2026.
- Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- Zhonghao Yang, Yu Li, Yanxu Zhu, Tianyi Zhou, Yuejin Xie, Haoyu Luo, Jing Shao, Xia Hu, and Dongrui Liu. Benchmarks for trajectory safety evaluation and diagnosis in openclaw and codex: Atbench-claw and atbench-codex. *arXiv preprint arXiv:2604.14858*, 2026.
- Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, et al. Claw-eval: Toward trustworthy evaluation of autonomous agents. *arXiv preprint arXiv:2604.06132*, 2026.
- Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pan, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, et al. A survey on trustworthy llm agents: Threats and countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 6216–6226, 2025.
- Hangtao Zhang, Chenyu Zhu, Xianlong Wang, Ziqi Zhou, Changgan Yin, Minghui Li, Lulu Xue, Yichen Wang, Shengshan Hu, Aishan Liu, et al. Badrobot: Jailbreaking embodied llm agents in the physical world. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Haochen Zhao and Shaoyang Cui. Clawtrap: A mitm-based red-teaming framework for real-world openclaw security evaluation. *arXiv preprint arXiv:2603.18762*, 2026.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2):1–124, 2023.
- Andy Zou, Maxwell Lin, Eliot Jones, Micha Nowak, Mateusz Dziemian, Nick Winter, Valent Nathanael, Ayla Croft, Xander Davies, Jai Patel, et al. Security challenges in ai agent deployment: Insights from a large scale public competition. *Advances in Neural Information Processing Systems*, 38, 2026.

A APPENDIX

You may include other additional sections here.